3S – Secure Streaming Solution (Part I): Secure Channel, Handshake and Flexible Cryptographic Configurations for Real-Time MPEG-4 Streaming

Constança Castro 79449 - p212418@campus.fct.unl.pt ;Tomás Ribeiro 72966 - tmgo.ribeiro@campus.fct.unl.pt
NOVA School of Science and Technology (FCT NOVA), Caparica, Portugal
Computer Systems Security, 2026/2027, 1st Semester – Work Assignment #1 – Pair Programming

Abstract. This report describes Part I of the Secure Streaming Solution (3S): a Secure Proxy and a Secure Stream Server that protect the streaming of MPEG-4 movies over TCP. A three-message handshake checks that both sides use the same cipher suite, binds each session to a fresh 256-bit challenge and to random sequence numbers, and carries the movie request. The Server then sends the movie as encrypted and authenticated blocks with consecutive sequence numbers, followed by an authenticated end block. The cipher suite is read from an XML file, and we tested eight configurations that use authenticated encryption (AES-GCM, AES-CCM, ChaCha20-Poly1305) or Encrypt-then-MAC (HMAC or AES-CMAC with AES, ChaCha20 or RC6). Offline tests and a man-in-the-middle on live streams show that an eavesdropper learns neither the movie nor the request, and that tampering, forgery, replay, reordering, dropping and truncation are detected. The protection adds 4.4% to 11.0% to the bytes sent, concurrent sessions do not interfere, and rate-based pacing at the bit rate of the movie gives smooth playback that starts in 1–2 s.

Keywords: secure streaming · authenticated encryption · Encrypt-then-MAC · replay protection · handshake · MPEG-4

1 Introduction

The baseline code of the assignment [1] has two programs: OriginSocketServer, which serves MP4 files in 1 KiB blocks over TCP, and ProxySocketServer, which relays them to a browser and answers HTTP Range requests (requests for a byte range of the file, used for seeking). The baseline sends everything in clear and offers no confidentiality, integrity, peer authentication or replay protection.

In Work Assignment 1 (WA1) we replace this link with a secure channel between the Secure Proxy (Proxy) and the Secure Stream Server (Server). The channel must provide confidentiality, integrity and authenticity of each message, and protection against replay and forgery, with a cipher suite that is negotiated by a handshake and set in configuration files, without recompilation. We implemented the configuration layer (eight valid configurations), the handshake, the protected message channel and its integration into the Proxy and the Server, with the metrics required by the assignment. We also wrote a testbench that attacks the channel offline and on live streams, tested concurrent sessions, and added optional playout pacing to the Proxy. The handshake uses the message formats of the assignment, which assume pre-shared keys. An authenticated Diffie-Hellman (DH) key exchange [2], and with it forward secrecy (past sessions stay secret if a long-term key leaks later), is left to Work Assignment 2 (WA2), as discussed in Section 3.

2 System Model and Architectural Design

The browser speaks plain HTTP to the Proxy, which runs on the trusted host of the viewer. For each request, the Proxy opens a TCP connection to the Server and runs the Secure Streaming Protocol (SSP), specified in Section 4. The Authentication Server belongs to WA2 and is not part of this work.

The code shared by both programs contains CryptoConfig (parsing and validation of the configuration), CryptoSuite (protection of one message), Handshake (message types 1–3), SecureChannel (framing of types 4, 5 and 255, sequence numbers, and additional authenticated data (AAD), which is authenticated but not encrypted) and exceptions mapped to alert codes. Two adapters connect the channel to the baseline code. In the Server, SecureOutputStream turns every write into a type 4 message and close() into the type 5 end block. In the Proxy, SecureInputStream returns only decrypted and verified data, and signals the end of the stream (-1) only after a valid type 5. With these adapters, the baseline code for HTTP parsing, Range handling and block relay is reused; only the stream it reads from or writes to changes.

3 Threat Model and Security Countermeasures

We assume a network attacker, Mallory, who controls the path between Proxy and Server: Mallory can eavesdrop, modify, inject, replay, reorder, duplicate and drop messages, and can open its own connections to the Server. We assume that both endpoints are not compromised, that the primitives (AES, ChaCha20, Poly1305, the GCM [3] and CCM [4] modes, HMAC-SHA2 [5], AES-CMAC [6], SHA-256) are secure when used correctly, and that the configured keys are secret and random. Endpoint compromise, attacks on the primitives and traffic analysis (message sizes and timing) are out of scope; implementation errors, weak configurations and protocol misuse are in scope.

Table 1 maps each threat to the mechanism that addresses it, and lists the requirements that Part I does not meet. An AEAD (authenticated encryption with associated data) mode encrypts and authenticates in one operation, a MAC (message authentication code) authenticates data with a secret key, a nonce is a value used only once, and ECB (electronic codebook) is a mode that encrypts equal blocks into equal ciphertext. SEQ denotes a sequence number and SRV the random challenge of the Server; the other message fields are defined in Section 4.

Table 1: Threat-to-mechanism mapping.

Threat	Mechanism	Status
Eavesdropping on media	AEAD or cipher+MAC; fresh random nonce per message. The HTTP request, response headers, Range and movie name also travel inside the encrypted channel	Mitigated (confirmed on a live stream, Section 7.4)
Tampering with a block	GCM/CCM/Poly1305 tag or MAC over aad‖nonce‖ct; any bit flip rejected (INTEGRITY_FAILURE)	Mitigated
Forgery	Requires the secret key(s); verification before use (MAC checked before decryption; constant-time compare)	Mitigated (given key secrecy)
Replay / duplication	Receiver accepts only the next expected SEQ; SEQX random (62 bit); fresh 256-bit SRV per handshake; R = SHA-256(SRV)	Mitigated
Reordering, dropping	Strict SEQ+1 rule; SEQ also in AAD, so rewriting the clear SEQ fails the tag	Mitigated
Truncation / block-END confusion	Authenticated end block (type 5); a connection closed before it is an error; version, release, type and SEQ in AAD	Mitigated (see note below the table)
Altered / downgraded proposal	Server accepts only a configuration identical to its own; confirmation CCC returned inside encrypted type 2 and compared by the Proxy	Mitigated
Tampering with HTTP metadata (status, Content-Length, Content-Range, Range)	Carried inside type 3 (Range) and inside the encrypted type 4 stream (response headers)	Mitigated
Weak configuration (ECB, no MAC, same key for cipher and MAC, wrong key size)	Rejected at load time by validate()	Mitigated
Resource exhaustion by forged size	Payload size capped at 1 MiB before allocation	Mitigated
Cross-session interference	Per-connection CryptoSuite, counters, SRV and SEQ; no shared mutable state between sessions	Mitigated (tested, Section 7.5)
Path traversal through the movie name	Server resolves the name against the media directory and answers 404 for anything outside it or not a regular file	Implemented; no dedicated test yet
Man-in-the-middle on key agreement	Peers authenticated only implicitly, by possession of the pre-shared key. No DH exchange in Part I (WA2)	Partial: no key agreement in Part I (out of scope, WA2)
Session-key compromise / forward secrecy	Keys are static: a leaked key exposes all recorded and future sessions	Not addressed: out of scope of Part I (WA2)
Forged alerts (type 255) / connection reset	Alerts are unauthenticated; an on-path attacker can already cut the connection	Out of scope
User authentication	Authentication Server	WA2

Note: if Mallory removes only the end block, all media has already been delivered and verified block by block; the Proxy still refuses to treat the stream as complete and logs it as INCOMPLETE (premature end of stream).

4 Specification of the Secure Streaming Protocol

Every message is Header‖Payload, where ‖ denotes concatenation. The header is sent in clear: version (2 B), release (2 B, currently 1:1), type (1 B) and payload size (4 B), 9 bytes in total. Sequence numbers have 8 bytes. E_K(·) denotes the configured protection of one message (Section 5), and aad_t = ver‖rel‖t is the AAD of a message of type t.

Handshake: The Proxy sends a random sequence number SEQX and its Proposed Cryptographic Configuration (PCC), read from proxycrypto.config. The Server compares the PCC with sservercrypto.config and, if they differ, answers with the alert CONFIG_REJECTED. Otherwise, it returns, encrypted, SEQY=SEQX+1, the Secure Random Value (SRV), a 256-bit challenge from SecureRandom, and the Cryptographic Configuration Confirmation (CCC). The Proxy checks SEQY and checks that the CCC equals its PCC. Since type 1 is sent in clear, this encrypted echo is how the Proxy detects a modified proposal. Message 3 carries SEQZ=SEQY+1, R=SHA-256(SRV) and the MEDIA field; the Server verifies R in constant time and only then starts the stream.

MEDIA field: Because browsers issue Range requests, MEDIA contains the movie name, a line break (CR LF) and the

Range header of the browser (empty if absent). It is encrypted and authenticated in type 3, so Mallory can neither learn which movie is requested nor change the requested range.

Streaming: Each block is sent as SEQB‖E_K(block), where SEQB starts at SEQZ+1 and is also authenticated as part of the AAD (aad_t‖SEQB). The receiver checks SEQB before it authenticates and decrypts the block. The first type 4 message carries the HTTP status line and headers produced by the Server (Content-Length, Content-Range, Content-Type), so they are protected like the media; the next type 4 messages carry the file in 1 KiB blocks. The last message is type 5, with SEQF in clear and SEQF‖"END"‖random encrypted. The receiver checks that both copies of SEQF match, and treats a connection that closes before a valid type 5 as an error. The Server also sends type 5 after a 404 answer, so an error response is also a complete, authenticated stream.

Alerts: A type 255 message carries a code (1 byte) and a reason: 1 unexpected message, 2 bad version, 3 malformed, 4 integrity failure, 5 bad sequence, 6 configuration rejected, 7 bad challenge, 8 media not found. Codes 1–7 are used as listed. A missing movie is answered with an HTTP 404 inside the authenticated stream (Section 6), so code 8 is defined but never sent. Every decryption failure produces the same alert, so the attacker cannot use the alert as an oracle to learn why a message failed (for example, its padding or its MAC).

5 Cryptographic Configuration Files

Both proxycrypto.config and sservercrypto.config follow the XML layout of the assignment [1] (SymCipherSuite, MACsuite, ConfiguredKeys), where NULL marks an unused field. Nothing cryptographic is hard-coded: the Java Cryptography Extension (JCE) transformation, the initialization vector (IV) size, the MAC name and the keys all come from the file. Keys are accepted in hexadecimal or Base64 and checked against their declared size. GenerateKey produces keys of the correct size for each algorithm, and ConfigTest prints how each file is interpreted (transformation, PCC, provider, MAC length) or why it is refused.

Message protection: In the AEAD modes (GCM [3], CCM [4], ChaCha20-Poly1305 [7]) a message is nonce‖ct‖tag, where ct is the ciphertext, and the AAD is passed to the cipher. The other configurations use Encrypt-then-MAC [8], that is, AES in counter (CTR) or cipher block chaining (CBC) mode, RC6 in CBC mode, or ChaCha20 alone, followed by a MAC with an independent key K2: nonce‖ct‖MAC_K2(aad‖nonce‖ct), where the MAC is an HMAC [5] or AES-CMAC [6]. A fresh random nonce is drawn for every message, and its size is taken from the file. Table 2 lists the eight configurations that we provide and test. AES-CCM, AES-CMAC and RC6 are not in the default JCE and use the Bouncy Castle (BC) provider [9], included in lib/; the other five need no external library.

Validation at load time: The loader refuses ECB, any non-AEAD mode without a MAC, a MAC without a key, a MAC key equal to the encryption key, key sizes other than 128, 192 or 256 bits, a key whose length differs from its declared size, a mode without an IV size, a CCM nonce outside 7–13 bytes, and ChaCha20 with a key that is not 256 bits. Five invalid example files (examples/INVALIDO-*.config) exercise these rules, and the loader refuses all five at start-up with a specific message.

The XML of the assignment is not well-formed (for example, <MACsuite> is closed by </MACSuite>), so we match tag names case-insensitively. The PCC format (ALG/MODE/IV/Padding) does not include the MAC, so a MAC mismatch between Proxy and Server is not refused with CONFIG_REJECTED; it is detected at the first protected message (INTEGRITY_FAILURE).

6 Secure Proxy and Stream Server: Design and Implementation

Proxy: At start-up, the Proxy loads and validates proxycrypto.config and prints its PCC. For each browser request, it parses the request line and the Range header, opens a TCP connection to the Server and runs Handshake.proxy() with the movie name and Range as MEDIA. It then reads from a SecureInputStream and reuses the baseline code to read the status line and headers (now decrypted) and to forward the body block by block to the browser. After the last byte of Content-Length, it requires a valid type 5 and treats any other data as a protocol error. On a protocol error, the Proxy sends the corresponding alert to the Server and aborts the stream; if no HTTP header has been sent yet, the browser receives 502 Bad Gateway. Since each block is written to the browser only after it has been verified and decrypted, the browser never receives unauthenticated data. A browser issues several Range requests while seeking and buffering, and each one is a separate SSP session with its own handshake, SRV and SEQX, so replay protection holds per session. The Proxy can pace the playout towards the browser (Section 7.5); by default, it does not pause.

Server: The Server accepts a connection, runs Handshake.server() with sservercrypto.config, splits MEDIA into movie name and Range, and replaces its output stream with a SecureOutputStream that starts at the first block SEQ returned by the handshake. The baseline code for headers and serveFile then runs unchanged, so Range handling (status 200, 206 and 416) still works. The Server resolves the movie name against the media directory and answers 404, through the secure stream, for any name outside that directory or that is not a regular file. The stream is closed in a finally block, which sends the authenticated type 5 in every case. Protocol errors are logged and answered with an alert. The baseline

already serves each connection in a thread of a cached pool, and we create the CryptoSuite and the counters per connection, so concurrent sessions share no state (Section 7.5).

Metrics: SecureChannel counts the bytes read from the network, and ProxyMetrics prints a report after every stream (each GET /video/… request is one stream). The report gives the HTTP status, whether the stream ended with a verified END, the number of encrypted blocks received and of decrypted blocks forwarded, the plaintext media bytes forwarded (compared with the size requested and with the original file), the bytes received from the network, and the throughput. We define throughput as the plaintext media bytes divided by the time from the end of the handshake (System.nanoTime) until the END is verified or the stream stops, with 1 kB = 1000 bytes. There is one more encrypted block than forwarded blocks, because the first type 4 message carries the HTTP headers: for the 25,941,346-byte test file there are 25,335 encrypted blocks, of which 25,334 carry media.

7 Validation

7.1 Testbench

We wrote the tests as Java programs in common/ and tools/ and saved their raw outputs in results/. CryptoSuiteTest checks the round trip, nonce uniqueness, tampering with every single bit, the AAD and truncation, and the overhead per 1 KiB message. ConfigTest loads every example file and prints its interpretation or the reason for refusal. In ChannelTest, Mallory manipulates a recorded session (type 2, three type 4 blocks, type 5) before it reaches a receiver. HandshakeTest uses real TCP sockets with a relay in the middle that applies each attack. StreamMallory is a live man-in-the-middle between the Proxy and the Server (the origin port of the Proxy points to Mallory) that applies one attack to the N-th type 4 message (default 50). In ConcurrencyTest, several HTTP clients, each through its own Proxy, download different movies at the same time and compare them with the originals by SHA-256. For end-to-end runs we used a web browser and curl against the Proxy, with a 25.9 MB MP4 file (dante.mp4) on the Server. The tests use the eight example configurations, or AES-GCM with a 256-bit key by default. We ran the measurements on two Windows PCs; byte counts do not depend on the machine, but throughputs do.

7.2 Results: cryptographic suite and channel

Cryptographic suite: In all eight configurations the round trip succeeds, each message gets a new nonce, and every modification is rejected. Table 2 lists the measured expansion of a 1 KiB message and the resulting wire overhead per block, which adds the 9 B header and the 8 B SEQ.

Table 2: Tested configurations: measured expansion of a 1 KiB message and derived per-block wire overhead.

Configuration	Nonce/IV	Suite expansion	Wire overhead per block
AES/GCM/NoPadding (96-bit IV)	12 B	+28 B (2.7%)	45 B (4.4%)
AES/CCM/NoPadding (BC)	12 B	+28 B (2.7%)	45 B (4.4%)
ChaCha20-Poly1305	12 B	+28 B (2.7%)	45 B (4.4%)
ChaCha20 + HMAC-SHA256	12 B	+44 B (4.3%)	61 B (6.0%)
AES/CTR + AES-CMAC	16 B	+32 B (3.1%)	49 B (4.8%)
AES/CTR + HMAC-SHA256	16 B	+48 B (4.7%)	65 B (6.3%)
RC6/CBC/PKCS5 + HMAC-SHA256 (BC)	16 B	+64 B (6.3%)	81 B (7.9%)
AES/CBC/PKCS5 + HMAC-SHA-512	16 B	+96 B (9.4%)	113 B (11.0%)

Channel attacks: We saved ChannelTest results for the five original configurations (AES-GCM, ChaCha20-Poly1305, ChaCha20+HMAC, AES-CTR+HMAC, AES-CBC+HMAC-SHA-512); in each, all 12 scenarios behave as required (60/60). The normal session is accepted; a replayed, swapped, dropped or injected-from-another-session block is rejected with BAD_SEQUENCE; a dropped block whose clear SEQs were “repaired”, a block turned into an end of stream (type 4→5) and a single flipped ciphertext bit are rejected with INTEGRITY_FAILURE; a changed version gives BAD_VERSION; a forged 2 GB size gives MALFORMED without allocating memory; and a missing end block or a cut connection is detected as a premature end of stream. The “repaired SEQ” case is detected only because SEQ is part of the AAD. The three newer configurations share the same channel code and are covered by CryptoSuiteTest, HandshakeTest and a full stream (Section 7.4).

7.3 Results: handshake

Handshake (real sockets): all 7 scenarios pass in each of the eight configurations: normal handshake (both sides obtain the same first-block SEQ and movie name); a Proxy proposing another algorithm (CONFIG_REJECTED); same configuration but different key (INTEGRITY_FAILURE); Mallory altering the PCC (CONFIG_REJECTED) or SEQX

(BAD_SEQUENCE); replay of old messages 1 and 3 (BAD_CHALLENGE, since SRV is fresh); and message 3 sent first (UNEXPECTED_MESSAGE).

We confirmed the same behavior end to end with the real programs. A Server configured with AES-GCM was contacted by Proxies configured with AES-CTR, AES-CBC, ChaCha20-Poly1305 and ChaCha20+HMAC. Both sides logged CONFIG_REJECTED, the browser received no media (0 bytes), and the Proxy answered the browser with 502 Bad Gateway. A Proxy with the right algorithm but a different key (aes-gcm-outra-chave.config) produced INTEGRITY_FAILURE on both sides in each of three attempts, again with no media forwarded.

7.4 Results: end-to-end streaming and live attacks

Full stream in all eight configurations: Table 3 shows one complete download of the 25,941,346-byte file through the browser-facing Proxy in each configuration. All eight ended with "COMPLETE (END verified)", and the media bytes forwarded equal the original file size. The measured network overhead matches the per-block cost derived in Table 2 (4.4% to 11.0%).

Table 3: End-to-end full-file stream, one run per configuration (loopback interface).

Configuration	Network bytes received	Overhead vs. file	Time	Throughput
AES/GCM/NoPadding	27,081,717	4.40%	1.573 s	16.5 MB/s
AES/CCM/NoPadding (BC)	27,081,717	4.40%	1.951 s	13.3 MB/s
ChaCha20-Poly1305	27,081,722	4.40%	1.256 s	20.7 MB/s
ChaCha20 + HMAC-SHA256	27,487,110	5.96%	1.362 s	19.0 MB/s
AES/CTR + AES-CMAC	27,183,066	4.79%	1.943 s	13.3 MB/s
AES/CTR + HMAC-SHA256	27,588,458	6.35%	1.843 s	14.1 MB/s
RC6/CBC/PKCS5 + HMAC-SHA256 (BC)	27,993,825	7.91%	2.138 s	12.1 MB/s
AES/CBC/PKCS5 + HMAC-SHA-512	28,804,609	11.04%	2.336 s	11.1 MB/s

Throughputs are single runs on the loopback interface (Section 7.1) and include the warm-up of the Java Virtual Machine, so the differences between configurations are within run-to-run noise. The lowest, 11.1 MB/s, is 49 times the 225.6 kB/s needed by the test video.

Browser playback and Range requests: With AES-GCM and with AES-CTR+HMAC, the browser played the video while we jumped to other positions in it, which generated 59 and 60 sessions respectively, each with its own handshake. Each session was a Range request answered with 206. One of them, for the final range of the file, ended with a verified END and exactly the 152,930 bytes requested. The others ended as "INCOMPLETE" with a connection-reset exception, because the browser abandons a request when it seeks or has buffered enough; the Proxy then stops without an integrity error. A request for a file that does not exist (sample.mp4, the default name in the test page) returned HTTP 404 inside a complete, authenticated stream of one block.

Live attacks with StreamMallory: Mallory sits between the real Proxy and the real Server and attacks the 50th type 4 message, which is the 49th block of media since the first message holds the HTTP headers. Table 4 gives the result of each attack as seen by the Proxy and by curl.

Table 4: Live attacks (AES-GCM, attack on message 50).

Attack	Proxy outcome	Media bytes delivered to the client
none (relay only)	COMPLETE, END verified	25,941,346 (all)
eavesdrop	COMPLETE, END verified	25,941,346 (all)
flip one byte of the ciphertext	INTEGRITY_FAILURE	49,152
drop the message	BAD_SEQUENCE (reordered or missing)	49,152
send the message twice	BAD_SEQUENCE (replayed or duplicated)	50,176
swap messages 50 and 51	BAD_SEQUENCE (reordered or missing)	49,152
change type 4 into type 5 in the header	INTEGRITY_FAILURE	49,152
inject a forged message with the next SEQ	INTEGRITY_FAILURE	49,152
remove the type 5 end message	INCOMPLETE (premature end of stream)	25,941,346 (all, no end marker)

In every active attack on a data block, the Proxy forwards only the blocks that arrived before the attack (48 media

blocks, 49,152 bytes; or 49 blocks, 50,176 bytes, when the message is duplicated, because the first copy is genuine), raises the matching alert and closes the connection. The client still receives status 200, because the headers had already been forwarded, but at most 50,176 of the 25,941,346 bytes announced in Content-Length, so the browser sees the truncation. These results match the offline ChannelTest. In the eavesdrop run, Mallory recorded the first megabyte in each direction (125 bytes Proxy→Server and about 1.05 MB Server→Proxy) and searched it for the movie name, the request line, the HTTP response line, the Content-Length header and the MP4 "ftyp" signature. None was found, whereas with the baseline all five appear in clear. The "cut" case is the one described in the note of Section 3.

7.5 Results: concurrency and playout pacing

Concurrent sessions: One Server (AES-GCM) and three Proxies (ports 8080, 8090 and 8091) served three different movies: dante.mp4 (25.9 MB), sintel.mp4 (7.6 MB) and caminandes.mp4 (126 MB). ConcurrencyTest ran the downloads first one after the other and then all at once. In every run (3 clients, and 6 clients in two runs) all files arrived with a SHA-256 equal to that of the original, and no Proxy log shows an ERROR line, so concurrent sessions do not interfere with each other's sequence numbers, challenges or keys. Running together took 1.94 s instead of 3.71 s for three clients (1.92× faster), and 2.55 s and 2.37 s instead of 5.10 s and 4.09 s for six (2.00× and 1.73×). The speed-up stays below the number of clients because the total time is bounded by the largest file (126 MB) and all processes share one machine.

Playout pacing: The baseline relays blocks without pausing (Thread.sleep(0)), and a comment in its code asks for the minimum pace that still gives good buffering and playout delay. We take that minimum as the average bit rate of the movie (size divided by duration): 225.6 kB/s for dante.mp4 (115 s), 146.3 kB/s for sintel.mp4 and 862.8 kB/s for caminandes.mp4. We first tested a fixed pause per 1 KiB block (-Ddelay). Table 5 shows that each block took about the pause plus 1.1 ms, and that a 10 ms pause took 15.6 ms, because Windows rounds sleeps up to its timer period. The largest pause that kept playback smooth was 3 ms for dante.mp4 (4 ms gave 197 kB/s and one stall) and 5 ms for sintel.mp4; caminandes.mp4 needs more than the 463.6 kB/s of a 1 ms pause, so it played smoothly only without a pause. We then added rate-based pacing (-Drate): the Proxy computes when each block is due at the target rate and sleeps only until then, so late wake-ups do not accumulate; -Drate=250 measured 250.1 kB/s. With this pacing alone, playback took about 30 s to start, because the browser first issues several Range requests to find the MP4 index at the end of the file. With -Dburst=2000, which sends the first 2,000 kB at full speed, playback started in 1–2 s and stayed smooth, and caminandes.mp4 played smoothly at 950 kB/s.

Table 5: Fixed pause per 1 KiB block and measured throughput (first 1 MB of dante.mp4, one run per pause).

Pause (ms)	0	1	2	3	4	5	6	8	10
kB/s	5188.6	463.6	323.5	247.9	197.3	167.4	132.9	113.1	65.9
ms per block	0.20	2.21	3.17	4.13	5.19	6.12	7.70	9.06	15.55

8 Conclusions and Future Work

Authenticating every block together with its sequence number, and ending each stream with an authenticated block, was enough to detect every manipulation we tried, offline and on live streams, before unauthenticated data reached the browser. The cost is 4.4% to 11.0% of the bytes sent: the AEAD modes add 45 B per block, while Encrypt-then-MAC adds from 49 B (AES-CTR with AES-CMAC) to 113 B (AES-CBC with HMAC-SHA-512), depending on the IV, the padding and the tag length. For playout, rate-based pacing with an initial burst kept the target rate within 0.1 kB/s and started playback in 1–2 s, which a fixed pause per block could not do.

The main limitation of Part I is the use of static pre-shared keys: there is no forward secrecy, and the peers are authenticated only by possession of the key. In WA2 we plan to add an authenticated ephemeral (EC)DH exchange and authenticated alerts.

References

[1] SSC teaching team. 3S – Secure Streaming Solution, Work Assignment 1: statement and baseline code. FCT NOVA, 2026.

[2] W. Diffie, M. Hellman. New Directions in Cryptography. IEEE Transactions on Information Theory 22(6), 1976.

[3] M. Dworkin. Recommendation for Block Cipher Modes of Operation: GCM and GMAC. NIST SP 800-38D, 2007.

[4] D. Whiting, R. Housley, N. Ferguson. Counter with CBC-MAC (CCM). RFC 3610, 2003.

[5] H. Krawczyk, M. Bellare, R. Canetti. HMAC: Keyed-Hashing for Message Authentication. RFC 2104, 1997.

[6] M. Dworkin. Recommendation for Block Cipher Modes of Operation: The CMAC Mode. NIST SP 800-38B, 2005.

[7] Y. Nir, A. Langley. ChaCha20 and Poly1305 for IETF Protocols. RFC 8439, 2018.

[8] M. Bellare, C. Namprempre. Authenticated Encryption: Relations among Notions and Analysis of the Generic Composition Paradigm. ASIACRYPT 2000, LNCS 1976, Springer.

[9] Legion of the Bouncy Castle. Bouncy Castle Crypto APIs for Java (bcprov-jdk18on 1.86). https://www.bouncycastle.org